Teoría de Algoritmos
(TB024) Curso Echevarría

Trabajo Práctico 1

24 de abril de 2026

Integrantes	Padrón
Joaquín Hernández	110470
Juan Francisco Cuevas	107963
Alexander Terrussi	107212
Dante Finci	00
Víctor Zacarías	107080

Índice

1. Ejercio #1: Comparación de Monedas **3**
1.1. Introducción y Problema . . . 3
1.2. Algoritmo de División y Conquista . . . 3
1.3. Complejidad temporal y cantidad de pesadas . . . 3
1.4. Metodología adoptadal . . . 4
1.5. Resultados y gráficos . . . 4
1.6. Relación entre tiempo de ejecución y pesadas . . . 4

2. Ejercicio #2: Algoritmo Paolo **5**
2.1. Introducción y problema . . . 5
2.2. Algoritmo . . . 5
2.3. Estructuras de datos . . . 7
2.4. Justificación Greedy . . . 7
2.5. Complejidad temporal y espacial . . . 7

3. Aspiradora con Backtracking **8**
3.1. Introducción y Problema . . . 8

4. Descomposición de Cadena con Programación Dinámica **9**
4.1. Introducción y Problema . . . 9
4.2. Supuestos . . . 9
4.3. Diseño del Algoritmo . . . 9
4.3.1. Definición del Subproblema . . . 9
4.3.2. Ecuación de Recurrencia . . . 9
4.3.3. Preprocesamiento de Palíndromos . . . 9
4.3.4. Subestructura Óptima . . . 9
4.3.5. Subproblemas Superpuestos . . . 9
4.3.6. Memoización . . . 10
4.3.7. Pseudocódigo . . . 10
4.4. Seguimiento . . . 10
4.5. Complejidad . . . 10
4.6. Sets de Datos . . . 11
4.7. Tiempos de Ejecución . . . 11
4.8. Resultados . . . 11
4.9. Resultados Experimentales . . . 11
4.10. Análisis de los Resultados . . . 12
4.11. Análisis del Desvío Estándar . . . 12
4.12. Comparación con la Curva Teórica . . . 12
4.13. Conclusión Experimental . . . 12
4.14. Conclusión . . . 12

1. Ejercio #1: Comparación de Monedas

1.1. Introducción y Problema

Se tiene una bolsa con n monedas de igual denominación, de las cuales exactamente una es falsa y es más liviana que el resto. El objetivo es identificar la moneda falsa sin ordenar por peso y usando División y Conquista.

La entrada se representa como una lista de pesos en un archivo de texto con valores separados por comas. Se asume que todos los pesos verdaderos son iguales (por ejemplo 10) y la falsa es más liviana (por ejemplo 4).

1.2. Algoritmo de División y Conquista

El algoritmo divide el arreglo en mitades. En cada paso compara las monedas de las posiciones centrales:

1. Si las dos monedas centrales son distintas, la más liviana es la falsa.
2. Si son iguales, se recurre sobre ambas mitades y luego se elige el mínimo entre los resultados.

Cada comparación de dos monedas se contabiliza como una pesada. La función principal parsea el archivo, llama al algoritmo y reporta tiempo y cantidad de pesadas.

El código del algoritmo de División y Conquista es:

```python
def encontrar_moneda_falsa_dyc(monedas, inicio, fin, contador):
    if inicio >= fin:
        return monedas[inicio]

    if fin - inicio == 1:
        contador[0] += 1
        return monedas[inicio] if monedas[inicio] <= monedas[fin] else monedas[fin]

    medio = (inicio + fin) // 2

    contador[0] += 1
    if monedas[medio] != monedas[medio + 1]:
        return monedas[medio] if monedas[medio] < monedas[medio + 1] else monedas[medio + 1]

    izquierdo = encontrar_moneda_falsa_dyc(monedas, inicio, medio, contador)
    derecho = encontrar_moneda_falsa_dyc(monedas, medio + 1, fin, contador)

    contador[0] += 1
    return izquierdo if izquierdo <= derecho else derecho
```

1.3. Complejidad temporal y cantidad de pesadas

Sea $T(n)$ el tiempo del algoritmo. La recurrencia es:

$$T(n) = 2T(n/2) + O(1)$$

La ecuación se justifica porque en cada llamada se divide el problema en dos subproblemas de tamaño $n/2$ (dos llamadas recursivas) y el costo adicional de comparar y seleccionar resultados es constante, es decir, $O(1)$.

Por el Teorema Maestro, $Complejidad = O(n)$.

1.4. Metodología adoptadal

Se evaluó el algoritmo con tamaños $n = \{100, 500, 1000, 5000, 10000, 50000\}$. Para cada tamaño se generó un archivo con una única moneda falsa y se ejecutó:

```
python3 ejercicios/ej-dyc/monedas.py 100
```

El tiempo se midió con la función time del sistema y se registró la cantidad de pesadas. Los resultados se guardaron en una tabla y se graficaron junto con una curva teórica lineal $O(n)$.

1.5. Resultados y gráficos

n	Tiempo (s)	Pesadas
100	0.000021	160
500	0.000080	746
1000	0.000162	1507
5000	0.000904	7945
10000	0.001782	15900
50000	0.010770	82764

Cuadro 1: Resultados experimentales del ejercicio 1.

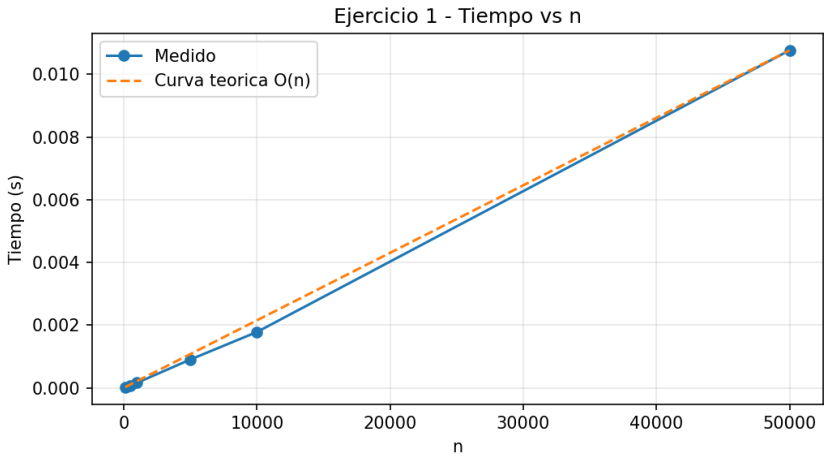

Figura 1: Tiempo de ejecución medido vs curva teórica $O(n)$.

Los gráficos muestran un crecimiento cercano al lineal, consistente con la complejidad analizada. La curva teórica se escaló para superponerla con la serie medida.

1.6. Relación entre tiempo de ejecución y pesadas

Es razonable vincular el tiempo con la cantidad de pesadas porque cada pesada implica una comparación de pesos, y el algoritmo realiza un número lineal de comparaciones. Sin embargo, el tiempo también incluye costos de lectura de datos y llamadas recursivas, por lo que la relación no es perfectamente proporcional. Aun así, ambos indicadores reflejan el mismo orden de crecimiento $O(n)$.

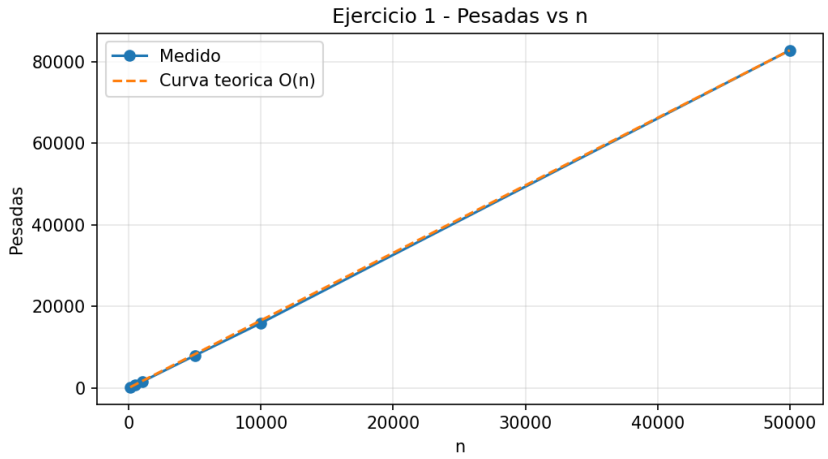

Figura 2: Cantidad de pesadas medida vs curva teórica $O(n)$.

2. Ejercicio #2: Algoritmo Paolo

2.1. Introducción y problema

El algoritmo "paolo", encontrado por el equipo de antropología, está escrito en un lenguaje inusual. El objetivo del ejercicio es identificar el problema que resuelve, reconocer el algoritmo que replica y justificar su carácter greedy, además de analizar el uso de estructuras de datos y la complejidad.

El problema que resuelve el código es el de caminos mínimos desde un único origen en un grafo no dirigido con pesos no negativos.

2.2. Algoritmo

El código original se presenta a continuación:

```
10 CLS
20 INPUT "INGRESE EL NUMERO DE VERTICES "; N
30 PRINT
40 DIM COST (N,N)
50 DIM DIST (N)
60 DIM SOL (N)
70 A=1
80 PRINT "INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)"
90 PRINT
100 INPUT "EL EJE "; A, B
110 IF A=0 THEN 140
120 INPUT "COSTO DEL EJE "; COST(A,B)
130 GOTO 100
140 FOR I=1 TO N
150 FOR J=1 TO N
160 IF COST (I,J)=0 THEN COST (I,J)=15000
170 NEXT J
180 NEXT I
190 FOR I=1 TO N
200 FOR J=1 TO I
210 COST (I,J)=COST (J,I)
220 NEXT J
230 NEXT I
240 INPUT "INGRESE EL VERTICE DE SALIDA "; V
250 FOR I=1 TO N
260 DIST(I)=COST(V,I)
270 SOL(I)=0
```

```
280 NEXT I
290 GOSUB 1000
300 PRINT "SALIDA","LLEGADA","DISTANCIA"
310 FOR I=1 TO N
320 IF DIST(I)<15000 THEN PRINT V, I, DIST(I)
330 NEXT I
340 INPUT "OTRA VEZ? (SI/NO) "; RES$
350 IF RES$="NO" THEN END
360 FOR I=1 TO N
370 SOL(I)=0: DIST(I)=0
380 NEXT I
390 GOTO 240
1000 SOL(V)=1
1010 DIST(V)=0
1020 FOR I=1 TO N-1
1030 U=15000
1040 FOR J=1 TO N
1050 IF DIST(J)<=U AND SOL(J)=0 THEN U=J
1060 NEXT J
1070 SOL(U)=1
1080 FOR J=1 TO N
1090 IF DIST(J)>= (DIST(U)+COST(U,J)) THEN DIST(J)=DIST(U)+COST(U,J)
1100 NEXT J
1110 NEXT I
1120 RETURN
```

Listing 1: Algoritmo Paolo original

Su traducción más literal a un lenguaje moderno (Python) es:

```

def paolo():
    n = int(input("INGRESE EL NUMERO DE VERTICES "))
    cost = [[0 for _ in range(n)] for _ in range(n)]
    dist = [0 for _ in range(n)]
    sol = [0 for _ in range(n)]
    a = 1
    print("INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)")
    print()
    while True:
        a, b = map(int, input("EL EJE ").split())
        if a == 0 and b == 0:
            break
        cost[a][b] = int(input("COSTO DEL EJE "))
    for i in range(n):
        for j in range(n):
            if cost[i][j] == 0:
                cost[i][j] = 15000
    for i in range(n):
        for j in range(i):
            cost[i][j] = cost[j][i]
    v = int(input("INGRESE EL VERTICE DE SALIDA "))
    for i in range(n):
        dist[i] = cost[v][i]
        sol[i] = 0
    paolo_sub(n, cost, dist, sol, v)
    print("SALIDA", "LLEGADA", "DISTANCIA")
    for i in range(n):
        if dist[i] < 15000:
            print(v, i, dist[i])
    res = input("OTRA VEZ? (SI/NO) ")
    if res == "NO":
        return
    for i in range(n):
        sol[i] = 0
        dist[i] = 0
    paolo()

def paolo_sub(n, cost, dist, sol, v):
    sol[v] = 1
    dist[v] = 0
```

```
    for i in range(n - 1):
        u = 15000
        for j in range(n):
            if dist[j] <= u and sol[j] == 0:
                u = j
        sol[u] = 1
        for j in range(n):
            if dist[j] >= (dist[u] + cost[u][j]):
                dist[j] = dist[u] + cost[u][j]

if __name__ == "__main__":
    paolo()
```

Listing 2: Algoritmo Paolo traducido

El algoritmo crea una matriz de adyacencia para representar un grafo, donde cada entrada $cost[i][j]$ almacena el costo de viajar desde el vértice i al vértice j. Si un costo no se ingresa, se asume un valor predeterminado de 15000 (considerado infinito en este contexto). Luego, la matriz se ajusta para que sea simétrica, lo que indica que el grafo es no dirigido.

A partir de un vértice de salida v, calcula las distancias más cortas a todos los demás vértices utilizando una variante del algoritmo de Dijkstra. En cada iteración, selecciona el vértice no resuelto con menor distancia y actualiza las distancias del resto usando ese vértice como punto de partida.

2.3. Estructuras de datos

Se utilizan una matriz de adyacencia y dos arreglos. La matriz *cost* permite consultar en tiempo constante el peso de cualquier arista, el arreglo *dist* guarda las mejores distancias encontradas hasta el momento, y el arreglo *sol* indica qué vértices ya fueron incorporados definitivamente a la solución.

2.4. Justificación Greedy

El algoritmo es greedy porque en cada paso toma una decisión local: elige el vértice no visitado con menor distancia estimada desde el origen. Esa regla greedy coincide con la estrategia central de Dijkstra para caminos mínimos con pesos no negativos.

2.5. Complejidad temporal y espacial

La complejidad temporal es $O(n^2)$ y la complejidad espacial es $O(n^2)$. El uso de memoria está dominado por la matriz de adyacencia de tamaño $n \times n$. En tiempo, el algoritmo realiza para cada uno de los n vértices una búsqueda lineal del siguiente vértice a fijar y una recorrida lineal para mejorar las distancias, resultando en un costo cuadrático.

3. Aspiradora con Backtracking

3.1. Introducción y Problema

Describir el problema de la aspiradora y el enfoque de Backtracking.

4. Descomposición de Cadena con Programación Dinámica

4.1. Introducción y Problema

El presente trabajo aborda el problema de descomponer una cadena en la menor cantidad posible de substrings que sean palíndromos. Se propone una solución basada en programación dinámica que permite resolver el problema en tiempo polinomial.

4.2. Supuestos

- La cadena de entrada tiene longitud $n \geq 1$.
- Una cadena de longitud 1 es un palíndromo.
- El acceso a caracteres es $O(1)$.
- Se dispone de memoria suficiente para almacenar estructuras $O(n^2)$.

4.3. Diseño del Algoritmo

4.3.1. Definición del Subproblema

Sea:

$$dp[i] = \text{mínima cantidad de palíndromos necesarios para } S[0..i]$$

4.3.2. Ecuación de Recurrencia

$$dp[i] = \min_{0 \leq j \leq i} \{dp[j-1] + 1 \mid S[j..i] \text{ es palíndromo}\}$$

Condición base:

$$dp[-1] = 0$$

4.3.3. Preprocesamiento de Palíndromos

Se define una matriz:

$$pal[i][j] = \begin{cases} True & \text{si } S[i..j] \text{ es palíndromo} \\ False & \text{en caso contrario} \end{cases}$$

$$pal[i][j] = (S[i] = S[j]) \wedge (j - i \leq 1 \vee pal[i+1][j-1])$$

4.3.4. Subestructura Óptima

La solución óptima para $S[0..i]$ depende de soluciones óptimas de subproblemas $S[0..j-1]$. Esto cumple con la propiedad de subestructura óptima.

4.3.5. Subproblemas Superpuestos

Los valores de $dp[i]$ y $pal[i][j]$ se reutilizan múltiples veces, evitando recomputaciones.

4.3.6. Memoización

Se utilizan:

- Matriz $pal[n][n]$
- Arreglo $dp[n]$

4.3.7. Pseudocódigo

```
func minPalindromos(S):
    n = len(S)
    crear matriz pal[n][n]

    para i en 0..n-1:
        pal[i][i] = True

    para length en 2..n:
        para i en 0..n-length:
            j = i + length - 1
            si S[i] == S[j] y (length == 2 o pal[i+1][j-1]):
                pal[i][j] = True

    crear dp[0..n-1]

    para i en 0..n-1:
        si pal[0][i]:
            dp[i] = 1
        sino:
            dp[i] = infinito
            para j en 1..i:
                si pal[j][i]:
                    dp[i] = min(dp[i], dp[j-1] + 1)

    devolver dp[n-1]
```

4.4. Seguimiento

Ejemplo: $S = ABA$

- $dp[0] = 1$
- $dp[1] = 2$
- $dp[2] = 1$

Resultado: 1 partición.

4.5. Complejidad

- Cálculo de palíndromos: $O(n^2)$
- Programación dinámica: $O(n^2)$

Complejidad total:

$$O(n^2)$$

4.6. Sets de Datos

Se utilizaron los siguientes tipos de datos:

- Mejor caso: cadenas repetidas (ej: AAAA)
- Peor caso: caracteres distintos (ej: ABCDEF)
- Casos aleatorios con semilla fija

4.7. Tiempos de Ejecución

Se midieron tiempos utilizando múltiples ejecuciones por cada tamaño de entrada, promediando los resultados y calculando el desvío estándar.

4.8. Resultados

Se obtuvo un crecimiento cuadrático en los tiempos de ejecución, consistente con la complejidad teórica.

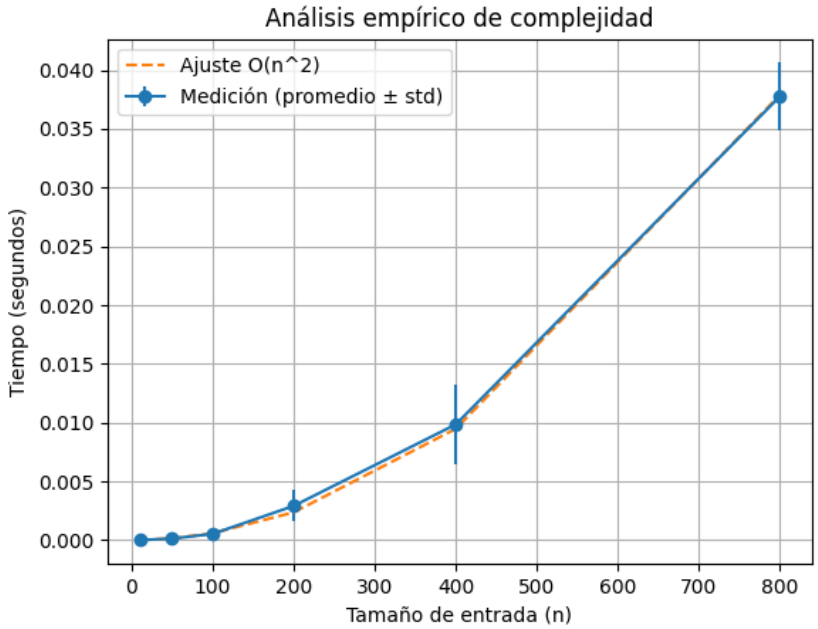

Figura 3: Comparación entre tiempos medidos y curva teórica

4.9. Resultados Experimentales

A continuación se presentan los resultados obtenidos a partir de la ejecución del algoritmo sobre distintos tamaños de entrada. Para cada tamaño, se midió el tiempo de ejecución promedio junto con su desvío estándar, a partir de múltiples ejecuciones.

n	Resultado	Tiempo Promedio (s)	Desvío Estándar
10	5	0.00000941	0.00000040
50	13	0.00013710	0.00004396
100	26	0.00053040	0.00009450
200	61	0.00291414	0.00132863
400	127	0.00985342	0.00342394
800	261	0.03777179	0.00285343

4.10. Análisis de los Resultados

A partir de los datos obtenidos, se observa que el tiempo de ejecución crece de manera consistente al aumentar el tamaño de la entrada. En particular, el crecimiento no es lineal, sino que se acelera a medida que n aumenta, lo cual es indicativo de un comportamiento cuadrático.

Este resultado es coherente con el análisis teórico del algoritmo, cuya complejidad temporal es $O(n^2)$. Si se comparan los tiempos entre distintas escalas, se puede notar que al duplicar el tamaño de la entrada, el tiempo de ejecución tiende a multiplicarse aproximadamente por un factor cercano a 4, lo cual coincide con el comportamiento esperado de una función cuadrática.

Por ejemplo, al pasar de $n = 200$ a $n = 400$, el tiempo crece de aproximadamente 0,0029 segundos a 0,0098 segundos, lo cual representa un incremento cercano a cuatro veces. De manera similar, al duplicar nuevamente hasta $n = 800$, el tiempo alcanza aproximadamente 0,0377 segundos, manteniendo la misma tendencia.

4.11. Análisis del Desvío Estándar

El desvío estándar permite evaluar la estabilidad de las mediciones realizadas. En los tamaños más pequeños, los valores son muy bajos, lo cual indica una alta consistencia en los tiempos medidos.

A medida que aumenta el tamaño de entrada, el desvío estándar también crece, lo cual es esperable debido a factores externos como la carga del sistema operativo, la gestión de memoria y otras tareas en ejecución. Sin embargo, en todos los casos, el desvío estándar se mantiene relativamente acotado en relación al tiempo promedio, lo que indica que las mediciones son confiables.

4.12. Comparación con la Curva Teórica

Al comparar los tiempos medidos con la curva teórica de complejidad $O(n^2)$, se observa que ambos presentan una tendencia similar. Esto valida empíricamente el análisis teórico realizado previamente.

Si bien pueden existir pequeñas desviaciones, especialmente en tamaños pequeños donde los costos constantes tienen mayor impacto, en general la curva experimental se ajusta adecuadamente al modelo teórico.

4.13. Conclusión Experimental

Los resultados obtenidos confirman que el algoritmo presenta un comportamiento cuadrático en la práctica, tal como se había deducido teóricamente. Esto demuestra la efectividad del enfoque de programación dinámica utilizado, tanto desde el punto de vista conceptual como en términos de rendimiento.

4.14. Conclusión

A lo largo de este trabajo se pudo ver cómo un problema que en principio parece complejo, como probar todas las formas posibles de dividir una cadena, puede resolverse de manera eficiente

utilizando programación dinámica.

La clave estuvo en cambiar la forma de pensar el problema: en lugar de evaluar todas las particiones posibles, se aprovechó la estructura del problema para construir la solución a partir de subproblemas más pequeños. Además, el preprocesamiento de palíndromos permitió evitar cálculos repetidos y mejorar significativamente el rendimiento.

En la práctica, los resultados obtenidos muestran que el tiempo de ejecución crece de manera cuadrática con el tamaño de la entrada, lo cual coincide con el análisis teórico realizado. Si bien para tamaños pequeños la diferencia no es tan evidente, a medida que crece el tamaño de la cadena se observa claramente esta tendencia.

En conclusión, el uso de programación dinámica permitió transformar un problema potencialmente exponencial en uno manejable, demostrando la importancia de identificar subestructuras óptimas y reutilizar resultados ya calculados.